GoalVault Financial DApp

Time locked savings with owner controlled withdrawals

SC6113 Individual Assignment | Implementation and evaluation | 10 October 2026

1 Introduction

GoalVault is a financial decentralized application that turns a savings intention into an enforceable release schedule. A user connects an Ethereum wallet, creates a goal, deposits test ETH and withdraws the recorded balance after a chosen time. The application combines a Solidity contract, a Python Flask backend and a responsive browser interface. It demonstrates programmable custody and verifiable transaction records without an administrator who can spend user deposits.

The implemented lifecycle has been executed on a local Ethereum-compatible chain. The evidence consists of 27 passing contract/backend tests, 13 passing browser checks, transaction receipts and screenshots. The browser automation uses an EIP-1193 test adapter; actual MetaMask extension interaction and Sepolia deployment remain separate manual acceptance steps. This distinction limits the claims made about deployment and usability.

2 Problem Statement

A student saving for a course or equipment purchase may want money reserved until a known date. A spreadsheet can track progress but cannot enforce the reservation. A custodial application can enforce restrictions, but its operator controls the ledger and may hold the funds. GoalVault explores a narrower alternative: a publicly inspectable contract enforces a rule agreed by the depositor, while only that depositor can request a release.

The design deliberately separates the target amount from the release condition. A user who saves 0.02 ETH toward a 0.05 ETH target can still recover the full 0.02 ETH after maturity. Requiring the target to be met could trap an underfunded goal indefinitely. Conversely, early withdrawal is unavailable: this makes the commitment meaningful but unsuitable for emergency reserves. No interest, purchasing-power protection or investment return is promised.

3 Objectives

Objective	Acceptance criterion
Owner control	A different wallet cannot deposit into or withdraw from a goal.
Enforced savings rule	Positive deposits are accepted before release; withdrawals are blocked until maturity.
Usable full-stack workflow	Connect a wallet, create a goal, deposit, withdraw and view confirmed history.
Verifiable implementation	Provide executable tests, real local-chain evidence and reproducible setup instructions.

Scope: native test ETH, one contract per configured network and independent goals per wallet. The original contribution is the integrated coursework implementation and explicit time-versus-target behavior, rather than a claim to invent a new DeFi primitive.

4 System Architecture

The contract is the authoritative record of ownership, balances and release times. The backend reads that record; it never signs transactions or receives private keys. The wallet authorizes each write independently. A blockchain transaction can therefore remain valid even if the read service later becomes unavailable, although the supplied interface depends on that service for configuration and display.

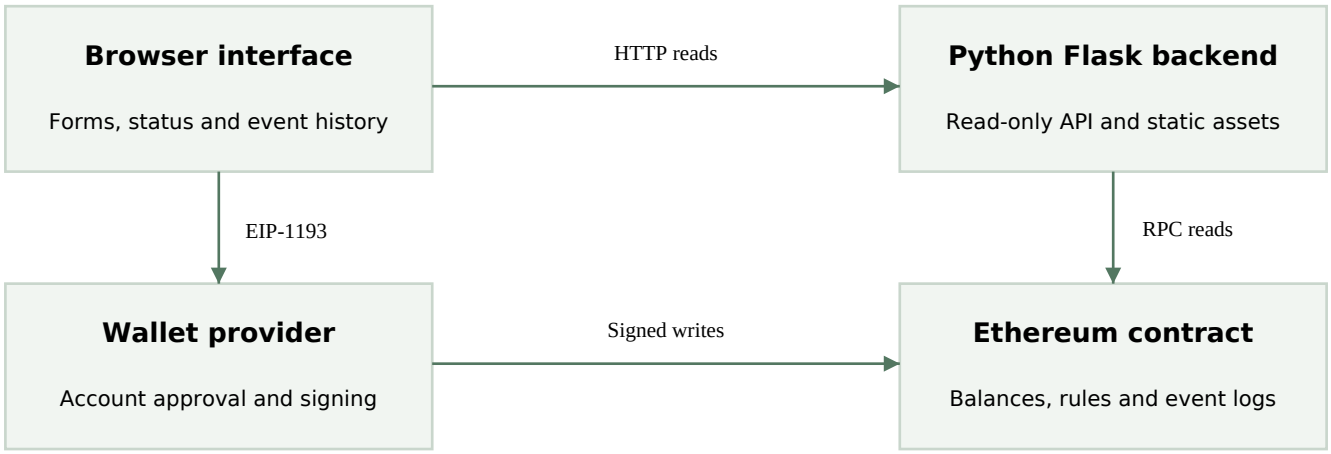


Figure 1. Read and write paths have different trust responsibilities. The wallet signs; the backend only reads.

For a deposit, the browser parses an ETH decimal into integer wei, asks the wallet to send deposit(id) with value, and shows the transaction hash while waiting for a receipt. After a successful mined receipt it reloads goals and event history through the backend. Public-chain deployments additionally link transaction hashes to the configured Sepolia explorer. A mined receipt is not represented as irreversible finality.

5 Technologies Used

Component	Implementation and purpose
Smart contract	Solidity 0.8.30; optimizer 200 runs; Shanghai EVM target; standalone source.
Web and backend	Python 3.12, Flask 3.1.2, web3.py 7.14.0 and Gunicorn 23.0.0; HTML/CSS/JS and ethers 6.15.0.
Wallet protocol	Injected EIP-1193 provider; account requests, chain checks and change events [2].
Local verification	Ganache 7.9.2, Node test runner and Playwright 1.56.1; local chain ID 31337.
Public test option	Sepolia, chain ID 11155111; manual Remix/MetaMask instructions supplied [4].

Render uses Gunicorn to serve Flask [6]. The included render.yaml specifies a free Python web service; a live URL has not yet been created.

No database is required: contract storage and event logs supply persistent records for a running chain. The local test chain is intentionally ephemeral. Ganache is pinned for reproducibility, but its repository is archived; migration to maintained tooling is a future maintenance task [5].

6 Smart Contract Design

Each Goal stores owner, label, target, balance, unlockAt and closed. A global ID identifies each goal; an owner-to-ID array enables bounded reads without scanning every user. totalLiabilities records the sum of all withdrawable goal balances. Owner checks use msg.sender, so backend requests cannot confer spending authority.

Function	Checks and state transition
createGoal	Label is 1–64 UTF-8 bytes; target > 0; release is 60 seconds to 365 days ahead. Store a new goal and emit GoalCreated.
deposit	Require owner, open goal, time before release and value > 0. Increase goal balance and total liabilities; emit Deposited.
withdraw	Require owner, open goal and maturity. Set balance to zero, close goal, reduce liabilities, emit Withdrawn and transfer the prior balance.

An open goal accepts deposits until its release timestamp. At maturity it becomes withdrawable without a separate state-changing transaction. After withdrawal it is permanently closed. An empty mature goal can also close. Targets are advisory, so overfunding is accepted and underfunding never blocks a mature withdrawal. There is no administrator, fee recipient, upgrade mechanism or early-release override.

Accounting and external calls

The tested invariant is that the sum of goal balances equals totalLiabilities, and contract ETH assets are at least that liability. Equality with actual ETH is not assumed because ETH can be forced into a contract independently of the deposit function. Ordinary unsolicited transfers revert. Forced surplus has no withdrawal path in this minimal design.

Withdrawal follows checks-effects-interactions: the contract validates authority and time, clears the balance and liability, then calls the owner. A shared reentrancy guard covers every state-changing entry point. If the recipient rejects ETH, TransferFailed reverts the complete transaction, including balance changes and events. The owner can retry if the receiving behavior is repaired. This implements the external-call precautions described in the Solidity guidance [1].

Validation and gas considerations

Custom errors identify invalid labels, targets, release times, owners, closed goals, zero deposits and failed transfers. Solidity checked arithmetic and integer wei avoid floating-point accounting. Release time uses block.timestamp, not the browser clock; the interface adds a time margin because transaction inclusion is not instantaneous.

All three write functions operate on a single goal without looping over all users. Short labels limit storage growth, and uint64 unlockAt shares a storage slot with the closed flag. Owner history is queried in pages. The optimizer is enabled, but no claim of globally optimal gas usage is made; the measured samples are reported in Section 9.

7 Application Design

The interface groups goal creation, savings progress and transaction history on one screen. A lock acknowledgement appears before goal creation, alongside a test-ETH and public-data notice. Progress cards show balance, target, release time and lifecycle state. Only actions appropriate to the chain snapshot are offered. The contract still validates every request independently.

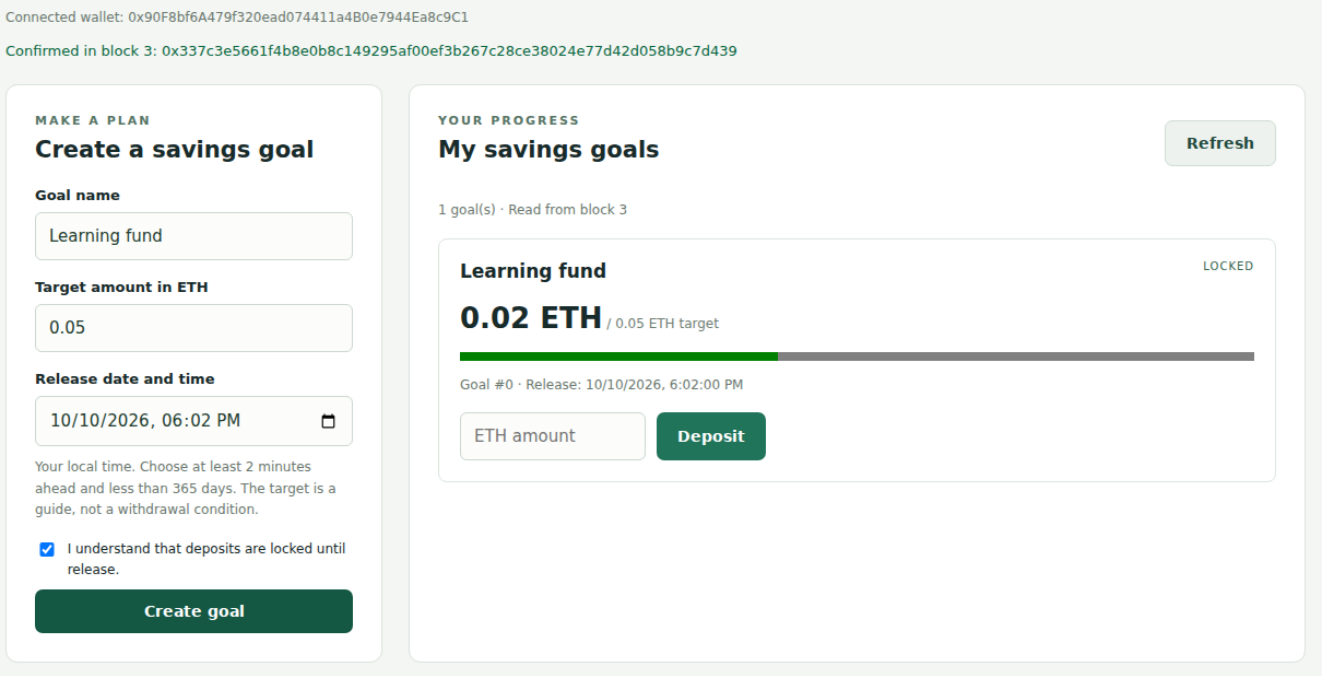


Figure 2. Actual local-chain browser run: a 0.05 ETH goal holds 0.02 ETH and remains locked. The automated wallet interface is a test adapter, not the MetaMask extension.

8 Implementation

The browser uses ethers BrowserProvider and a wallet-backed signer [3]. It checks the configured chain before connection and before sending transactions. Account, chain and disconnect events clear stale state and require reconnection. Amounts reject negatives, exponent notation and more than 18 decimal places. Goal labels are rendered as text nodes, preventing label content from becoming executable HTML.

The transaction status distinguishes wallet approval, pending hash, mined success and failure. Wallet rejection returns an actionable message. A two-minute confirmation timeout warns that a transaction may still be pending, discouraging accidental resubmission. Replacement transactions can resolve to the replacement receipt; a confirmed write is distinguished from a later display-refresh failure.

The Flask backend verifies chain ID and exact compiled runtime bytecode before returning blockchain data. Its GET-only APIs validate addresses and range parameters, return at most 20 goals per page, and read each goal page at one block snapshot. History filters contract logs by the owner topic over windows of up to 2,000 blocks, with an older-range cursor. RPC errors return HTTP 503 without exposing credentials. The server supplies security headers and serves ethers locally, avoiding a runtime CDN dependency.

9 Testing and Results

Tests ran against a local EVM with Solidity 0.8.30, Node.js 24.19.0 and Python 3.12. The contract/backend suites passed 27 of 27 cases; the browser suite passed 13 of 13 checks. These are functional checks, not an independent security audit or evidence of public-network performance.

Coverage	Observed result and evidence
Contract lifecycle and inputs	18 contract tests passed: deployment, ownership, bounds, deposits, maturity, closing, failed transfers, reentrancy and balance invariants.
Backend integration	9 Flask tests passed: live reads, owner-filtered events, input validation, read-only routes, assets, headers and readiness failures.
Browser workflow	13 Flask/Gunicorn browser checks passed, including create/deposit/withdraw, missing wallet, rejection, wrong chain, invalid amounts, reload and account change.
Responsive layout	A 390px viewport had no horizontal page overflow; a full mobile screenshot is included. No user study was conducted.
Manual acceptance	Real MetaMask prompts and Sepolia deployment were not executed. README provides the remaining verification procedure.

The browser run created goal #0 in block 2, deposited 0.02 ETH in block 3, advanced the local clock, and withdrew in block 5. The final balance was zero and the goal was closed. The target remained 0.05 ETH, demonstrating that maturity, rather than target completion, governs withdrawal. Reloading rebuilt history from chain logs.

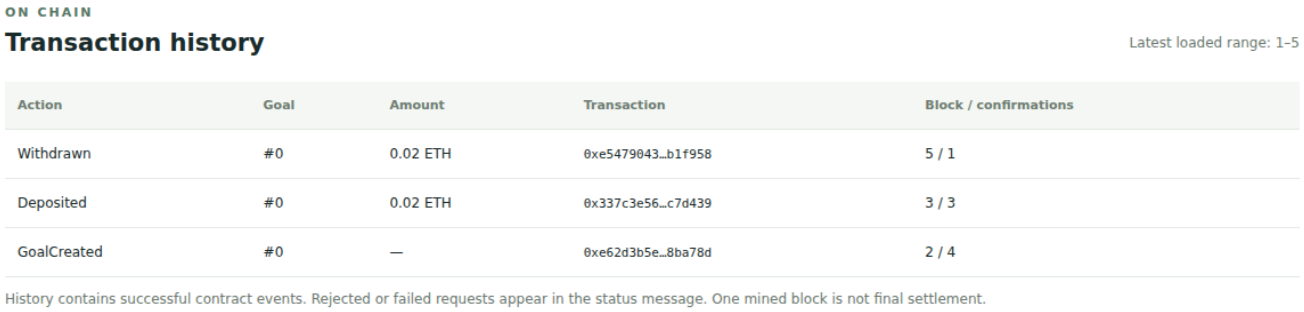


Figure 3. Actual local event history after withdrawal. Local time advancement is a test facility and is not available to application users on Sepolia. Full-size screenshots and transaction hashes are retained in evidence/.

Sample transaction	Gas units
Deployment	738,103
Create	152,079
Deposit	74,887
Withdraw	45,423

Gas values are individual successful receipts from the contract test run, not averages. Create is the final helper-created goal; deposit and withdrawal are the named recorded samples in contract.test.mjs. Costs vary with storage state and label length. Local mining speed was not used to estimate Sepolia latency or throughput.

10 Challenges Encountered

Timing and reproducible tests

A test initially scheduled a goal exactly 60 seconds after the latest block. By the time the transaction was mined, the next block could have advanced by a second, making that timestamp too short. Lifecycle tests were changed to use a 120-second margin; exact minimum and maximum bounds are checked through simulation against the current block. The interface likewise requires a margin rather than promising a boundary-time transaction will succeed.

Separating financial state from display state

A wallet can change accounts or networks while a read is in flight. The frontend increments a session generation on change and ignores stale read responses. It also disables write buttons during a transaction. Monetary values stay as bigint or decimal strings through the contract/API path; conversion to a Number is limited to visual progress and timestamps, not transfer amounts.

Making failure evidence meaningful

Input validation alone does not prove that contract rules hold. Tests therefore send an actual early-withdrawal transaction with an explicit gas limit and verify that its mined receipt reverts without releasing funds. Separate malicious receiver contracts attempt a callback withdrawal or reject the ETH transfer. The tests verify single payment and complete accounting rollback, respectively.

11 Limitations

The current evidence establishes the local functional lifecycle and browser-provider integration. It does not establish successful use of the real MetaMask extension, public RPC reliability, Sepolia finality, or deployed HTTPS hosting. The supplied manual procedure is therefore an acceptance task rather than a completed result. No questionnaire or real-user usability study was performed.

GoalVault provides no interest or stable fiat value. ETH volatility, transaction fees, lost keys and irreversible lock commitments limit financial suitability. It has no emergency unlock, recovery administrator or insurance. Contract wallets that cannot receive ETH may be unable to withdraw. Public labels and balances are visible to others; this design offers transparency rather than financial confidentiality.

The API still depends on a trusted display/configuration service and RPC provider. Event windows reduce query size but are not a durable reorganization-aware indexer. The UI does not persist pending or failed transaction history across reloads. Production rate limiting, load testing, dependency hardening and an independent audit are not complete. A passing test suite cannot exclude untested vulnerabilities.

12 Future Improvements

The first priority is to complete real-wallet and Sepolia acceptance with deployment receipts and screenshots. Next steps are maintained local-chain tooling, fuzzed accounting invariants, a reorganization-aware event index with durable pending-status tracking, and public-hosting operational controls. ERC-20 support or optional recovery rules would require a separate threat model and contract version; adding them casually would change the custody promise.

13 Conclusion

GoalVault implements a complete local financial DApp lifecycle with contract-enforced release rules, wallet-authorized writes and a backend that reconstructs state and history from the blockchain. The central design decision is that a savings target measures progress while time controls withdrawal. This avoids trapping an underfunded goal while preserving the agreed lock period.

The local evidence supports the implemented ownership rules, accounting, failure handling and end-to-end web workflow: 27 contract/backend tests and 13 browser checks passed. The result remains a coursework prototype. Actual MetaMask interaction, Sepolia deployment and the Render URL remain pending. Repository access and workspace confirmation are required to deploy.

Reproducibility and evidence index

File or command	Purpose
pip install -r requirements.txt; npm ci; npm run compile	Install Python and Node dependencies; reproduce ABI and bytecode.
npm run chain; npm run deploy:local; python app.py	Start the ephemeral local chain, deploy the contract and serve the DApp.
npm test; npm run test:ui	Execute isolated contract/API tests and browser lifecycle checks.
evidence/test-results.txt	18 contract tests plus 9 passing Python Flask integration tests.
evidence/browser-results.json	13 browser checks with an explicit local test-provider scope.
evidence/ui-chain-history.json	Actual deployment and create/deposit/withdraw transaction hashes.
README.md and START_HERE_CN.md	Setup, MetaMask connection, Sepolia deployment and remaining acceptance steps.

The source includes .sol, standalone abi.json, app.py, requirements.txt, HTML/CSS/JS, render.yaml, tests and screenshots. No database or video is needed. Full-size evidence images supplement the cropped figures in this report. The test adapter is confined to the test harness; the application itself expects an injected wallet provider.

References

[1] Solidity contributors. Security Considerations.
https://docs.soliditylang.org/en/latest/security-considerations.html

[2] Ethereum Improvement Proposals. EIP-1193 Ethereum Provider JavaScript API.
https://eips.ethereum.org/EIPS/eip-1193

[3] ethers. Version 6 Providers documentation.
https://docs.ethers.org/v6/api/providers/

[4] Ethereum.org. Networks.
https://ethereum.org/en/developers/docs/networks/

[5] ConsenSys archive. Ganache repository and project status.
https://github.com/ConsenSys-archive/ganache

[6] Render. Deploy a Flask App on Render.
https://render.com/docs/deploy-flask

All web references accessed 10 October 2026. Implementation-specific claims are supported by the supplied source files and execution evidence, not by the reference documents.